

Production Grade Library Management System (Java Full-Stack)

**Library Management System (Java Full-
Stack)**

▼ Example Json Data

▼ Books

```
{ "id": 1, "isbn": "978-3-16-148410-0", "title": "The Art of  
Java Development", "author": "Ashok Zarmariya", "genre":  
"PROGRAMMING", "publisher": "Zosh Publications",  
"publicationDate": "2023-05-15", "language": "English",  
"pages": 420, "description": "A comprehensive guide to  
mastering Java programming, covering modern practices,  
frameworks, and real-world applications.", "totalCopies": 50,  
"availableCopies": 30, "price": 499.99, "coverImageUrl":  
"https://example.com/images/the-art-of-java-development.jpg",  
"active": true }
```

▼ Book Loan

▼ genre

```
{ "code": "FICTION", "name": "Fiction", "description": "Genre
that includes imaginative or invented stories, often
exploring characters, plots, and themes not based on real
events.", "displayOrder": 1, "active": true, "parentGenreId":
null, "parentGenreName": null }, // for function books
"subGenres": [ { "code": "MANAGEMENT", "name": "Management",
"description": "Books on strategic planning, leadership, and
organizational management principles.", "displayOrder": 1,
"active": true, "parentGenreId": 1 }, { "code":
"HUMAN_RESOURCES", "name": "Human Resources", "description":
"Covers employee management, organizational behavior, and HR
development topics.", "displayOrder": 2, "active": true,
"parentGenreId": 1 }, { "code": "FINANCE", "name": "Finance",
"description": "Includes books related to corporate finance,
investment, accounting, and financial planning.",
"displayOrder": 3, "active": true, "parentGenreId": 1 }, {
"code": "MARKETING", "name": "Marketing", "description":
"Focuses on marketing strategies, consumer behavior,
branding, and advertising techniques.", "displayOrder": 4,
"active": true, "parentGenreId": 1 }, { "code": "OPERATIONS",
"name": "Operations", "description": "Covers production
management, logistics, supply chain optimization, and process
improvement.", "displayOrder": 5, "active": true,
"parentGenreId": 1 } ]
```

▼ subscription plan

```
{ "id": 1, "planCode": "GOLD_001", "name": "Gold Membership",
  "description": "Ideal for avid readers – extended checkout
duration and more book access.", "durationDays": 90, "price":
999, "currency": "INR", "maxBooksAllowed": 5,
"maxDaysPerBook": 21, "displayOrder": 1, "isActive": true,
"isFeatured": true, "badgeText": "Most Popular",
"adminNotes": "This plan is performing well in mid-tier
subscribers.", "priceInMajorUnits": 999.0,
"monthlyEquivalentPrice": 333.0, "createdAt": "2025-10-
10T12:00:00", "updatedAt": "2025-10-10T12:00:00",
"createdBy": "admin", "updatedBy": "admin" } { "id": 2,
"planCode": "PLATINUM_001", "name": "Platinum Membership",
"description": "Premium access with maximum borrowing limits,
faster checkouts, and priority support.", "durationDays":
180, "price": 1799, "currency": "INR", "maxBooksAllowed": 10,
"maxDaysPerBook": 30, "displayOrder": 2, "isActive": true,
"isFeatured": true, "badgeText": "Best Value", "adminNotes":
"Top-tier subscription with strong retention.",
"priceInMajorUnits": 1799.0, "monthlyEquivalentPrice":
299.83, "createdAt": "2025-10-10T12:00:00", "updatedAt":
"2025-10-10T12:00:00", "createdBy": "admin", "updatedBy":
"admin" }
```

▼ LLD

▼ Book

Overview

The Book Module is the core component of the Library Management System that handles the complete lifecycle of books in the library catalog. It provides comprehensive functionality for managing book inventory, searching, and tracking availability.

Features

1. Book Catalog Management

- Create single or multiple books (bulk operations)
- Update book information
- Soft delete (mark as inactive) or permanently delete books
- Track book availability and copies

2. Search & Discovery

- Simple search by ID or ISBN
- Advanced search with multiple filters (title, author, genre, availability)
- Pagination and sorting support
- Text-based search across title, author, and ISBN

3. Inventory Tracking

- Total copies management
- Available copies tracking
- Automatic validation to ensure available copies don't exceed total copies

4. Statistics & Analytics

- Total active books count
 - Total available books count
 - Real-time inventory insights
-



Data Model

Book Entity

Table Name: `books`

Field	Type	Required	Constraints	Description
<code>id</code>	Long	Auto	Primary Key	Unique book identifier
<code>isbn</code>	String(20)	✓	Unique, ISBN format	International Standard Book Number
<code>title</code>	String(255)	✓	1-255 chars	Book title
<code>author</code>	String(255)	✓	1-255 chars	Author name
<code>genre</code>	Genre	✓	Foreign Key	Book genre/category
<code>publisher</code>	String(100)	✗	Max 100 chars	Publisher name
<code>publicationDate</code>	LocalDate	✗	-	Date of publication
<code>language</code>	String(20)	✗	Max 20 chars	Book language
<code>pages</code>	Integer	✗	1-50000	Number of pages
<code>description</code>	String(2000)	✗	Max 2000 chars	Book description
<code>totalCopies</code>	Integer	✓	≥ 0	Total copies owned
<code>availableCopies</code>	Integer	✓	≥ 0 , $\leq$ totalCopies	Copies available for borrowing
<code>price</code>	BigDecimal(10, 2)	✗	≥ 0 , 8 digits.2 decimals	Book price
<code>coverImageUrl</code>	String(500)	✗	Max 500 chars	Cover image URL
<code>active</code>	Boolean	✓	Default: true	Soft delete flag
<code>createdAt</code>	LocalDateTime	Auto	Immutable	Creation timestamp
<code>updatedAt</code>	LocalDateTime	Auto	-	Last update timestamp

Relationships

- **Book** → **Genre**: Many-to-One relationship (each book belongs to one genre)
-

API Endpoints

Base URL

`/api/books`

1. Create a Book

Endpoint: `POST /api/books`

Description: Create a new book in the catalog

► Request Body:

2. Create Books in Bulk

Endpoint: `POST /api/books/bulk`

Description: Create multiple books in a single request (batch operation)

► Request Body:

3. Get Book by ID

Endpoint: `GET /api/books/{id}`

Description: Retrieve a specific book by its ID

Example: `GET /api/books/1`

4. Get Book by ISBN

Endpoint: `GET /api/books/isbn/{isbn}`

Description: Retrieve a book using its ISBN number

Example: `GET /api/books/isbn/978-0-13-468599-1`

5. Update Book

Endpoint: `PUT /api/books/{id}`

Description: Update an existing book's information

Example: `PUT /api/books/1`

► **Request Body:**

6. Soft Delete Book

Endpoint: `DELETE /api/books/{id}`

Description: Soft delete a book (marks as inactive, doesn't remove from database)

Example: `DELETE /api/books/1`

Response: `200 OK`

json

```
{
  "message": "Book deleted successfully",
  "success": true
}
```

Note: Soft-deleted books have `active = false` and won't appear in default searches.

7. Permanently Delete Book

Endpoint: `DELETE /api/books/{id}/permanent`

Description: Permanently remove a book from the database

Example: `DELETE /api/books/1/permanent`

Response: `200 OK`

json

```
{  
  "message": "Book permanently deleted",  
  "success": true  
}
```

⚠ **Warning:** This action cannot be undone!

8. List/Search Books (Simple)

Endpoint: `GET /api/books`

Description: List all books with optional filters using query parameters

9. Advanced Search

Endpoint: `POST /api/books/search`

Description: Perform complex search with multiple criteria. Use POST for combining text search with filters.

Validation Rules

Required Fields

- ISBN (must be valid format)
- Title (1-255 characters)
- Author (1-255 characters)
- Genre ID
- Total Copies (≥ 0)
- Available Copies (≥ 0 , must be $\leq$ total copies)

Optional Fields

- Publisher (max 100 characters)
- Publication Date
- Language (max 20 characters)
- Pages (1-50,000)
- Description (max 2000 characters)

- Price (≥ 0 , max 8 integer digits, 2 decimal places)
- Cover Image URL (max 500 characters)

Business Rules

1. **ISBN Uniqueness:** No two books can have the same ISBN
 2. **Copy Validation:** Available copies cannot exceed total copies
 3. **Soft Delete:** Deleted books remain in database but are marked inactive
 4. **Automatic Timestamps:** Created and updated timestamps are managed automatically
-

Error Handling

Common Error Responses

400 Bad Request - Validation errors

json

```
{  
  "message": "ISBN is mandatory",  
  "success": false  
}
```

404 Not Found - Book doesn't exist

json

```
{  
  "message": "Book not found with id: 123",  
  "success": false  
}
```

409 Conflict - ISBN already exists

json

```
{  
  "message": "Book with ISBN 978-0-13-468599-1 already exists",  
  "success": false  
}
```

▼ Genera

GENRE MODULE DOCUMENTATION (Entity + Controller)

Overview

The **Genre module** manages library book categorization.

It supports:

- ✓ Hierarchical genres (Parent → Child → Sub-child ...)
- ✓ Unlimited sub-genre depth (tree structure)
- ✓ Soft & hard delete support
- ✓ Public search API + pagination + sorting
- ✓ Genre activation management for display filtering

This enables libraries to manage flexible, multidimensional category trees like:

```
Fiction |— Mystery | |— Detective Fiction | |— Cozy Myster  
y |— Fantasy |— Drama
```

 **Data Model:** **Genre**

Field	Type	Constraints	Description
<code>id</code>	Long	PK, Auto Increment	Unique identifier
<code>code</code>	String	Required, Unique, 2-50 chars, <code>^[A-Z_]+\$</code>	System code in uppercase (used for mapping/search)
<code>name</code>	String	Required, 2-100 chars	Public display name
<code>description</code>	String	Optional max 500 chars	Details about genre
<code>displayOrder</code>	Integer	Default <code>0</code>	Ordering for UI sorting
<code>active</code>	Boolean	Default <code>true</code>	Used to soft-disable category
<code>parentGenre</code>	Genre (Many-to-One)	Nullable	Shows if category belongs to another
<code>subGenres</code>	List<Genre>	Auto managed	Recursive hierarchy children
<code>books</code>	List<Book>	Reference only	Books belonging to genre
<code>createdAt</code>	LocalDateTime	Auto-timestamp	Creation audit
<code>updatedAt</code>	LocalDateTime	Auto-timestamp	Update audit

Notable Features

Feature	Explanation
Recursive structure	Genre → Sub-Genres → Sub-Sub-Genres
Helper methods	<code>addSubGenre()</code> and <code>removeSubGenre()</code> maintain relationships
Soft delete support	Through active flag & controller endpoints
High-performance querying	Indexed fields → <code>code</code> , <code>name</code> , <code>active</code>

API CONTROLLER — `/api/genres`

1 Create Genre

Method	Endpoint	Auth
<code>POST</code>	<code>/api/genres</code>	Public/Admin (based on implementation)

Body Example

```
{ "code": "MYSTERY", "name": "Mystery", "description": "Suspense & investigation stories", "displayOrder": 1, "active": true, "parentGenreId": null }
```

Response: `201 CREATED → GenreDTO`

2 Create Bulk Genres

Method	Endpoint
<code>POST</code>	<code>/api/genres/bulk</code>

Used to insert multiple related genres at once.

3 Get Genre by Code

| GET | `/api/genres/code/{code}` |

```
GET /api/genres/code/FANTASY
```

Returns `GenreDTO`, throws `GenreException` if missing.

4 Update Genre

| PUT | `/api/genres/{id}` |

Updates code, name, description, parent relationships, etc.

5 Delete Genre (Soft Delete)

| DELETE | `/api/genres/{id}` |

Marks inactive → Does *not* remove DB record.

6 Hard Delete Genre (Permanent)

| DELETE | `/api/genres/{id}/hard` |

⚠ Irreversible + books using it may break!

QUERY ENDPOINTS

Endpoint	Description
<code>GET /api/genres/active</code>	Returns flat list of all active genres
<code>GET /api/genres/active/hierarchy</code>	Returns only parent genres recursively with subtrees
<code>GET /api/genres/top-level</code>	Fetch only root-level genres (no parent)
<code>GET /api/genres/{parentId}/sub-genres</code>	Get children of specific genre

Example Response (hierarchy):

```
[ { "name": "Fiction", "subGenres": [ { "name": "Mystery", "subGenres": [ { "name": "Detective Fiction" }, { "name": "Cozy Mystery" } ] } ] } ]
```

SEARCH + PAGINATION

Endpoint	Example
<code>GET /api/genres/search</code>	<code>/api/genres/search?term=fiction&page=0&size=10&sortBy=name&sortDir=ASC</code>

Returns `Page<GenreDTO>`.

STATISTICS ENDPOINTS

▼

Endpoint	Returns
GET /api/genres/count	Total active genres
GET /api/genres/{id}/book-count	How many books use this genre
GET /api/genres/{id}/in-use	Boolean → Genre linked to books?



Book Loan (Checkout Management)



Overview



Final Notes

Book Loan Module handles the entire borrowing lifecycle of a book — from checkout to return and overdue handling.

Strength	Benefit
It enables users to borrow books while the system tracks due dates, renewals, overdue status, and fines automatically.	
Recursive Genre Tree	Complex library categorization supported
Admins can manage user loans, update records, and analyze borrowing trends.	
Bulk creation	Faster onboarding / data migration
Indexed search + Pagination	Scales for 10K+ genres
Fit for real life	Flexible lifecycle control



Features

1. Book Borrowing (Checkout)

- Users and admins can checkout books
- Admins can issue books to any user
- Validates book availability before issuing
- Automatically assigns due date based on policies

2. Book Return (Check-In)

- Users/admins can return books
- Return updates loan status and availability
- Tracks overdue return and fines calculation

3. Renewal Management

- Borrowed books can be renewed until renewal limit
- Validation for overdue and renewal limit
- Automatically extends due date

4. Loan Tracking & Queries

- Get loan details by loan ID
- User-wise and Admin-level listing
- Filter by status (ACTIVE, OVERDUE, RETURNED etc.)
- Pagination and searching with filters

5. Overdue Tracking & Fine Handling

- Marks overdue automatically via scheduler/admin trigger
- Tracks overdue days dynamically
- Fine tracking supported via linked Fine entity collection

6. Loan Analytics & Reports

- Admin dashboard-level statistics
 - Loan distribution, overdue count, active loan count
-



Data Model



BookLoan Entity

Table Name: `book_loans`

Field	Type	Required	Constraints	Description
<code>id</code>	Long	Auto	PK	Unique loan identifier
<code>user</code>	User	✓	FK <code>user_id</code>	Borrower of the book
<code>book</code>	Book	✓	FK <code>book_id</code>	Book issued in the loan
<code>type</code>	BookLoanType	✓	ENUM	Loan type (NORMAL, SHORT_TERM, etc.)
<code>status</code>	BookLoanStatus	✓	ENUM	Current state (CHECKED_OUT, RETURNED, OVERDUE)
<code>checkoutDate</code>	LocalDate	✓	-	Date book was issued
<code>dueDate</code>	LocalDate	✓	-	Date book must be returned
<code>returnDate</code>	LocalDate	✗	Nullable	Actual return date
<code>renewalCount</code>	Integer	✗	Default: 0	Number of renewals used
<code>maxRenewals</code>	Integer	✗	Default: 2	Maximum allowed renewals
<code>finer</code>	List<Fine>	✗	One-To-Many	Associated unpaid/paid fines
<code>notes</code>	String(500)	✗	Max 500 chars	Additional remarks
<code>isOverdue</code>	Boolean	✗	Default false	Marks if loan exceeded due date

<code>overdueDays</code>	Integer	✗	Default 0	Total late days
<code>createdAt</code>	LocalDateTime	Auto	-	Record creation timestamp
<code>updatedAt</code>	LocalDateTime	Auto	-	Last modification timestamp

Relationships

Entity	Relation	Description
BookLoan → User	Many-to-One	One user may have multiple active/past loans
BookLoan → Book	Many-to-One	A book may have multiple loans over time
BookLoan → Fine	One-To-Many	Multiple fines can be attached to one loan

API Endpoints

Base URL

`/api/book-loans`

1. Checkout Book (Logged-in User)

POST `/api/book-loans/checkout`

- Checks out a book for the currently authenticated user

Request Body Example

```
{ "bookId": 14, "dueDays": 14 }
```

Response

```
{ "id": 22, "status": "CHECKED_OUT", "dueDate": "2025-01-30"
}
```

2. Checkout Book For Another User (Admin)

POST `/api/book-loans/checkout/user/{userId}`

Example Request

```
{ "bookId": 14, "dueDays": 10 }
```

3. Return (Check-In) Book

POST `/api/book-loans/checkin`

Marks loan as returned and processes overdue/fines if applicable.

Request

```
{ "bookLoanId": 22 }
```

4. Renew Borrowed Book

POST `/api/book-loans/renew`

Extends due date if renewal is allowed.

Example

```
{ "bookLoanId": 22, "extendDays": 7 }
```

5. Get Loan By ID

GET `/api/book-loans/{id}`

6. Get My Book Loans (User/Admin)

GET `/api/book-loans/my?status=ACTIVE&page=0&size=20`

Supports pagination + status filtering

7. Get Loans of User (Admin)

GET `/api/book-loans/user/{userId}?status=OVERDUE&page=0&size=20`

8. Advanced Search (Admin)

POST `/api/book-loans/search`

Filter by date range, status, user, overdue, type etc.

9. Update Loan Record (Admin)

PUT `/api/book-loans/{id}`

10. Mark Overdue Loans (Admin Task)

POST `/api/book-loans/admin/update-overdue`

Automatically marks loans past due.

11. Loan Statistics (Admin)

GET `/api/book-loans/statistics`

Returns checkout statistics useful for dashboards.

Validation Rules

Required Inputs

Field	Rule
User	Must exist & be valid
Book	Must exist & be available
checkoutDate	Required at creation
dueDate	Cannot be before checkoutDate
status	Must be valid Enum

Business Rules

1. Only active (CHECKED_OUT) loans can be renewed or returned
 2. Cannot renew if overdue or renewalCount == maxRenewals
 3. Overdue detection updates `isOverdue` and `overdueDays`
 4. Fines associated remain traceable even after return
-

Error Handling

Status	Reason	Example Response
400 Bad Request	Validation or business rule failed	<code>{ "message": "Book already issued" }</code>
404 Not Found	Loan not found	<code>{ "message": "BookLoan not found: 90" }</code>
409 Conflict	Renewal/Checkout violation	<code>{ "message": "Cannot renew overdue loan" }</code>
500 Server Error	Internal failure	<code>{ "message": "Failed to update overdue loans" }</code>

▼ Book Review

BOOK REVIEW SYSTEM — MODEL + API DOCUMENTATION

Review + Rating + Helpful Votes + Reader Verification

This module enables users to post book reviews with ratings, verify eligibility based on completed loans, update/delete reviews, view analytics & handle helpful votes.

Key Objectives

Feature	Description
 Rating System	Users rate books 1–5
 Review Text	Title + long review content
 Verification	Only readers with completed loan can review
 Ownership Rules	Only author can update/delete their review
 Helpful Metric	Users can mark reviews as helpful
 Statistics	Average rating + distribution + total reviews

ENTITY — **BookReview**

Field	Type	Constraints	Description
<code>id</code>	Long	Primary Key	Unique review ID
<code>user</code>	User	<code>@NotNull</code> , FK	User who posted review
<code>book</code>	Book	<code>@NotNull</code> , FK	Reviewed book
<code>rating</code>	Integer	<code>1-5</code> , Required	Star rating
<code>reviewText</code>	String	<code>10-2000 chars</code> , Required	Review content
<code>title</code>	String	<code>≤200 chars</code> , Optional	Headline of review
<code>isVerifiedReader</code>	Boolean	Default= <code>false</code>	True if user finished book loan
<code>isActive</code>	Boolean	Default= <code>true</code>	Soft delete flag
<code>helpfulCount</code>	Integer	Default= <code>0</code>	Total helpful votes
<code>createdAt</code>	LocalDateTime	Auto timestamp	Created date
<code>updatedAt</code>	LocalDateTime	Auto timestamp	Updated date

Indexes & Constraints

Name	Fields	Purpose
<code>idx_book_id</code>	<code>book_id</code>	Fast book-review lookup
<code>idx_user_id</code>	<code>user_id</code>	Review history per user
<code>idx_rating</code>	<code>rating</code>	Rating-based queries
<code>uk_user_book_review</code>	<code>(user_id, book_id)</code>	1 review per user per book

REST API — `BookReviewController`

Base URL:

/api/reviews

1 Create Review

Only users who have **completed a book loan** may post a review.

Method	Endpoint	Auth
POST	/api/reviews	USER / ADMIN

◆ Request Body

```
{ "bookId": 1, "rating": 5, "title": "Amazing Read!", "reviewText": "Very insightful and well written. Highly recommended!" }
```

◆ Responses

Status	Meaning
201 CREATED	Review successfully created
400 BAD REQUEST	Invalid rating or user not eligible

2 Update Review (Only Author)

Method	Endpoint	Auth
PUT	/api/reviews/{reviewId}	USER / ADMIN

```
{ "rating": 4, "title": "Updated Title", "reviewText": "Still good, but not perfect." }
```

3 Soft Delete Review

Method	Endpoint	Auth
DELETE	/api/reviews/{reviewId}	USER / ADMIN

Returns success message.

4 Get Review By ID

Method	Endpoint
GET	/api/reviews/{reviewId}

Returns review or 404 NOT FOUND .

Retrieve Reviews For Book

Method	Endpoint
GET	/api/reviews/book/{bookId}

Query Parameters

Param	Values	Default
filter	ALL / BY_RATING / VERIFIED_ONLY / TOP_HELPFUL	ALL
rating	1–5 (if filter=BY_RATING)	—
page	number	0
size	number	10

 Examples

```
GET /api/reviews/book/10 GET /api/reviews/book/10?filter=BY_RATING&rating=5 GET /api/reviews/book/10?filter=TOP_HELPFUL
```

User Review History

Method	Endpoint	Access
GET	/api/reviews/my-reviews	USER/ADMIN
GET	/api/reviews/user/{userId}	PUBLIC

Paginated responses.

Book Rating Statistics

Method	Endpoint
GET	/api/reviews/book/{bookId}/statistics

Returns:

```
{ "bookId": 1, "bookTitle": "Effective Java", "averageRating": 4.6, "totalReviews": 120, "ratingDistribution": { "5": 80, "4": 30, "3": 8, "2": 2, "1": 0 }, "verifiedReaderReviews": 95 }
```

Helpful Vote on Review

Method	Endpoint	Auth
POST	/api/reviews/{reviewId}/helpful	USER/ADMIN

Increases `helpfulCount` & returns updated review.

Check Review Eligibility

Method	Endpoint	Auth
GET	/api/reviews/can-review/{bookId}	USER/ADMIN

```
{ "canReview": true }
```

Admin Analytics

Method	Endpoint	Role
GET	/api/reviews/admin/statistics	ADMIN

```
{ "totalReviews": 892 }
```

▼ Fine

Fine — Fine Management System

Overview

The **Fine** entity manages monetary penalties associated with book loans. Unlike a loan-bound structure, fines are separated for detailed financial monitoring, historical auditability, multi-fine association per loan, and independent payment/waiver handling.

Key Features

Feature	Description
Multiple fines allowed per loan	Enables tracking overdue, damage, handling fees independently
Full & Partial payment support	Outstanding balance calculated dynamically
Waiver workflow	Admin-controlled waiving with timestamp & reason
Real-time fine status	Tracks <code>PENDING</code> , <code>PARTIALLY_PAID</code> , <code>PAID</code> , <code>WAIVED</code>
Audit fields	Includes <code>createdAt</code> & <code>updatedAt</code> with Hibernate timestamps
Transaction tracking	Stores payment identifiers & processing admin

◆ Data Model — *Fine Entity*

Field	Type	Constraints	Description
<code>id</code>	Long	Primary Key	Unique identifier for fine
<code>bookLoan</code>	BookLoan	NotNull, @ManyToOne	Loan associated with fine
<code>user</code>	User	NotNull, @ManyToOne	User responsible for fine
<code>type</code>	FineType (ENUM)	NotNull	OVERDUE, DAMAGE, LOST, etc.
<code>amount</code>	Long	NotNull	Total fine amount
<code>amountPaid</code>	Long	Default=0	Total paid towards fine
<code>status</code>	FineStatus (ENUM)	NotNull, default=PENDING	Current payment state
<code>reason</code>	String	500 chars	Why fine was issued
<code>notes</code>	String	1000 chars	Staff notes/admin remarks
<code>waivedBy</code>	User	Nullable	Admin who waived fine
<code>waivedAt</code>	DateTime	Nullable	When waiver occurred
<code>waiverReason</code>	String	500 chars	Waiver justification
<code>paidAt</code>	DateTime	Nullable	Final payment timestamp
<code>processedBy</code>	User	Nullable	Payment processing staff
<code>transactionId</code>	String	100 chars	External payment reference
<code>createdAt</code>	DateTime	Auto generated	Record creation timestamp

<code>updatedAt</code>	DateTime	Auto updated	Last modification time
------------------------	----------	--------------	------------------------

Business Logic Summary

Method	Behavior
<code>getAmountOutstanding()</code>	Returns <code>amount - amountPaid</code>
<code>isFullyPaid()</code>	Checks if paid >= amount or status=PAID
<code>isWaived()</code>	True if status=WAIVED
<code>isPending()</code>	True if PENDING or PARTIALLY_PAID
<code>applyPayment(amount)</code>	Updates paid, status, paidAt when complete
<code>waive(admin, reason)</code>	Sets WAIVED, assigns admin & timestamps



API — Fine Controller

Base URL:

`/api/fines`

1 Create Fine (Admin)

Method	Endpoint
POST	<code>/api/fines</code>

Body Example

```
{ "bookLoanId": 12, "userId": 8, "type": "OVERDUE", "amount": 150, "reason": "Returned 5 days late" }
```

2 Fine Payment

✓ Full Payment

```
POST /api/fines/{id}/pay?transactionId=TXN_44HA8
```

Returns **PaymentInitiateResponse** or error.

3 Waive Fine (Admin)

```
POST /api/fines/waive
```

Body Example

```
{ "fineId": 22, "waiverReason": "Lost due to system error" }
```

4 Fine Retrieval

Use Case	Method	Endpoint
Get fine by ID	GET	<code>/api/fines/{id}</code>
Get fines by loan	GET	<code>/api/fines/book-loan/{bookLoanId}</code>
Get my fines (filterable)	GET	<code>/api/fines/my?status=PENDING&type=OVERDUE</code>
Admin — All fines paginated	GET	<code>/api/fines?status=&type=&userId=&page=0&size=20</code>

5 Aggregations / Statistics

Endpoint	Description
<code>GET /api/fines/my/total-unpaid</code>	Total unpaid balance (current user)
<code>GET /api/fines/statistics/user/{userId}/unpaid</code>	Admin — total dues for a user
<code>GET /api/fines/statistics/collected</code>	Total PAID fines
<code>GET /api/fines/statistics/outstanding</code>	Total unpaid system-wide
<code>GET /api/fines/statistics/user/{userId}/has-unpaid</code>	Does user owe money?

6 Delete Fine (Admin)

`DELETE /api/fines/{id}`

- ▼  Irreversible — recommended only for auditing corrections.

Response DTOs

RESERVATION MODULE

DOCUMENTATION (Entity + Controller)

```
{ "total": 400 } // TotalFinesResponse { "hasUnpaidFines": true } // HasUnpaidFinesResponse
```

Overview

The **Reservation/Hold module** manages user requests for books that are currently unavailable.

It implements automated queueing, expiration rules, fulfillment tracking and pickup window management.

Supports:

- ✓ Multi-user waiting queue per book
- ✓ Automatic reservation lifecycle transitions
- ✓ Expiry handling based on availability time
- ✓ Queue-position lookup
- ✓ Full advanced search & history filtering
- ✓ Admin + authenticated user separation



Data Model — Reservation

Field	Type	Constraints	Description
<code>id</code>	Long	PK, Auto-generated	Reservation identifier
<code>user</code>	User	NotNull, Many-to-One LAZY	User who placed reservation
<code>book</code>	Book	NotNull, Many-to-One LAZY	Target reserved book
<code>status</code>	ReservationStatus (Enum)	Default: <code>PENDING</code>	State in lifecycle (PENDING/AVAILABLE/FULFILLED/EXPIRED/CANCELLED)
<code>reservedAt</code>	LocalDateTime	Required	When reservation was created
<code>availableAt</code>	LocalDateTime	Nullable	When book became available for pickup
<code>availableUntil</code>	LocalDateTime	Nullable	Pickup deadline (48–72 hrs typical)
<code>fulfilledAt</code>	LocalDateTime	Nullable	When user checkout fulfilled reservation
<code>cancelledAt</code>	LocalDateTime	Nullable	When reservation cancelled or expired
<code>queuePosition</code>	Integer	Nullable	User rank in waiting queue
<code>notificationSent</code>	Boolean	Default false	Whether pickup email sent to user
<code>notes</code>	String TEXT	Optional	Cancellation reason or admin remarks
<code>createdAt</code>	Timestamp	Auto	Creation audit
<code>updatedAt</code>	Timestamp	Auto	Modification audit

🔥 Special Utility Methods

Method	Returns	Intent
<code>isActive()</code>	<code>true</code> if PENDING or AVAILABLE	Helps display only valid reservations
<code>canBeCancelled()</code>	Same logic as above	Conditional UI button control
<code>hasExpired()</code>	Detects expiry via <code>availableUntil</code>	Used for auto-expiry background jobs

🚦 Reservation Lifecycle Flow

User requests reservation → PENDING Book returned → AVAILABLE (email sent) User picks up → FULFILLED User doesn't pick up in time → EXPIRED User cancels manually → CANCELLED

Queue moves automatically when top user fulfills/cancels.

🎮 REST Controller — `/api/reservations`

1 Create Reservation — (Authenticated User)

Method	POST
Endpoint	<code>/api/reservations</code>
Body	<code>ReservationRequest</code>
Returns	<code>ReservationDTO</code>

Used when logged-in user reserves a book.

2 Create Reservation for Specific User — (Admin)

| POST | `/api/reservations/user/{userId}` |

Allows librarian to place holds manually.

3 Cancel Reservation

| DELETE | `/api/reservations/{id}` |

- Cancels only if status is `PENDING` or `AVAILABLE`.
 - Updates queue automatically inside service.
-

4 Fulfill Reservation (Check-out)

| POST | `/api/reservations/{id}/fulfill` |

Marks reservation as **FULFILLED** + removes from queue.

5 Get Reservation by ID

| GET | `/api/reservations/{id}` |

Returns detailed DTO or 404 if not found.

USER-SCOPED QUERIES

6 Get My Reservations (Filter + Pagination)

| GET | `/api/reservations/my` |

Query Params:

```
status=PENDING|AVAILABLE|FULFILLED|CANCELLED|EXPIRED activeOnly=true/false page=0&size=20 sortBy=reservedAt&sortDirection=DESC
```

Response → `PageResponse<ReservationDTO>`

7 Advanced Body-Based Search (User)

| POST | `/api/reservations/my/search` |

Accepts `ReservationSearchRequest` body

Supports complete filter, sorting, timeframe queries.

ADMIN-LEVEL QUERIES

8 Search All Reservations (Query Params)

| GET | `/api/reservations` |

Params:

```
userId, bookId, status, activeOnly page, size, sortBy, sortDirection
```

9 Advanced Search (POST Body)

| POST | `/api/reservations/search` |

Best for dashboard UI filters.

12 Queue Position Check

| GET | `/api/reservations/{id}/queue-position` |

Response:

```
{ "queuePosition": 2, "message": "There are 1 person(s) ahead of you" }
```

Automatically returns contextual text.



Performance & Index Structure

Index	Benefit
<code>idx_user_id</code>	Fast per-user history
<code>idx_book_id</code>	Queue & next-user lookup O(1)
<code>idx_status</code>	Instant filtering for pending/active
<code>idx_reserved_at</code>	Sorted timeline queries
<code>idx_available_until</code>	Batch expiry sweeps & cron tasks

Strengths of Module

Feature	Operational Benefit
Queue-based reservation	Handles high-demand books smoothly
Expiry + auto-removal design	Frees books if user doesn't come
Notification tracking	Prevents duplicate emailing
Multi-filter admin search	Works even at 100k+ reservation rows
DTO protected API layer	Zero exposure of entity internals

▼ Subscription

SUBSCRIPTION MODULE DOCUMENTATION (Entity + Controller)

Overview

The **Subscription module** manages paid library access for users.

It links each user to a subscription plan defining:

- ✓ Borrow limits
- ✓ Loan duration per book
- ✓ Plan pricing + currency
- ✓ Auto-renew + cancellation workflow
- ✓ Activation via payment gateway

It also supports full history retrieval and admin-level management of expired or active plans.

Data Model — **Subscription**

Field	Type	Constraint	Description
<code>id</code>	Long	PK	Unique subscription identifier
<code>user</code>	User	NotNull, ManyToOne LAZY	Subscription owner
<code>plan</code>	SubscriptionPlan	NotNull, ManyToOne EAGER	Base plan (duration, price, limits)
<code>planName</code>	String	Cached	Helps preserve history even if plan changes later
<code>planCode</code>	String	Cached	Used for invoice mapping + plan lineage
<code>price</code>	Long	Stored minor currency unit	e.g. 99900 = 999.00 rupees
<code>currency</code>	String(3)	ISO code	e.g. INR, USD, EUR
<code>startDate</code>	LocalDate	Required	Subscription activation date
<code>endDate</code>	LocalDate	Required	Auto calculated via plan duration
<code>isActive</code>	Boolean	Default = TRUE	Determines borrow permission
<code>maxBooksAllowed</code>	Integer	Required + Positive	Concurrent borrow limit
<code>maxDaysPerBook</code>	Integer	Required + Positive	Loan duration per book
<code>autoRenew</code>	Boolean	Default = FALSE	Enables recurring billing
<code>cancelledAt</code>	LocalDateTime	Optional	Timestamp for manual cancellation
<code>cancellationReason</code>	Text	Optional	Stored justification or admin note

<code>notes</code>	Text	Optional	Internal comments
<code>createdAt</code>	Timestamp	Auto	Audit creation trace
<code>updatedAt</code>	Timestamp	Auto	Audit modification trace

🔥 Subscription Logic Inside Entity

Method	Purpose
<code>isValid()</code>	Ensures subscription active + date in range
<code>isExpired()</code>	Check via <code>endDate < today</code>
<code>getDaysRemaining()</code>	Countdown for UX reminders
<code>calculateEndDate()</code>	Computes <code>endDate = startDate + durationDays</code>
<code>initializeFromPlan()</code>	Loads pricing + quota from base plan
<code>getPriceInMajorUnits()</code>	Converts 99900 → 999.00 (UI safe)
<code>canBorrowMoreBooks(x)</code>	Enforces borrowing cap

🚦 Subscription Lifecycle

`subscribe()` → payment processing → activation webhook → ACTIVE
 ACTIVE → expiration hits → EXPIRED (auto or admin)
 ACTIVE → user cancels → CANCELLED
 EXPIRED → `renewSubscription()` → ACTIVE reset

🎮 REST Controller —

`/api/subscriptions`

1 Create Subscription (Payment Initiation)

POST	<code>/api/subscriptions/subscribe</code>
Body → <code>SubscribeRequest</code>	
Returns → <code>PaymentInitiateResponse</code>	

Triggers payment gateway + temp subscription record.

2 Get User Subscription History

| GET | `/api/subscriptions/history?userId={id(optional)}` |

Returns list of `SubscriptionDTO`

Includes expired, cancelled, active, renewed — full timeline.

3 Get Active Subscription

| GET | `/api/subscriptions/user/active?userId=x` |

Returns one subscription — the currently valid one.

Throws 404 if user has none.

4 Check Valid Subscription (Boolean)

| GET | `/api/subscriptions/check?userId=x` |

Example Response:

```
{ "message": "User has valid subscription", "success": true }
```

Used before allowing book issuance.

5 Get Subscription by ID

| GET | `/api/subscriptions/{id}` |

Secured for USER + ADMIN.

Throws error if subscription does not exist.

6 Renew Subscription

| POST | `/api/subscriptions/renew/{subscriptionId}` |

Re-triggers payment → extends plan → recalculates endDate.

7 Cancel Subscription

| POST | `/api/subscriptions/cancel/{subscriptionId}?reason=` |

Stores:

- `cancelledAt`
- `cancellationReason`
- sets `isActive = false`

Borrow rights stop immediately.

8 Activate Subscription (Webhook Callback)

| POST | `/api/subscriptions/activate?subscriptionId=x&paymentId=y` |

Used after successful payment confirmation.

ADMIN ENDPOINTS

9 List all Active Subscriptions (Paginated)

| GET | `/api/subscriptions/admin/active?page=0&size=20` |

Good for dashboard reporting + renewal follow-ups.

10 Deactivate Expired Subscriptions (Manual or CRON Job)

| POST | `/api/subscriptions/admin/deactivate-expired` |

Automatically sets expired ones to inactive.

Use in scheduled task e.g. daily at midnight.



Table Index Advantages

Index	Purpose
<code>idx_user_id</code>	Fast lookup per user
<code>idx_is_active</code>	Quick count of active users
<code>idx_end_date</code>	Efficient expiry batch processing
<code>idx_plan_id</code>	Reporting per subscription plan



Module Strengths

Feature	Benefit
Cached plan metadata	Billing remains historically accurate
Auto expiry & prorated renewal	No manual admin workload
Auto-renew compatible	Payment gateway future integration ready
Borrow rights controlled at entity level	Prevents backend fraud / bypass

▼ Subscription Plan

Overview

The **Subscription Plan Module** manages all membership plans available for users. It provides complete lifecycle features such as creating, updating, activating/deactivating plans, tracking billing cycles, trial periods, pricing, and access levels. This module acts as the backbone of the subscription infrastructure and integrates with payment, user subscription, and billing services.

Features

1. Plan Management

- Create single or multiple subscription plans
- Update plan details, pricing, validity, usage limitations
- Soft delete (mark inactive) or permanently delete plans
- Attach perks and role-based access controls to plans

2. Pricing & Billing

- Support for monthly/yearly billing cycles
- Trial period configuration for new users
- Auto-renewal configuration for recurring subscriptions
- Supports one-time purchase and recurring plans

3. Search & Filtering

- Search by Plan ID or Title
- Filter by billing cycle, price range, active status, etc.
- Pagination for large plan lists

4. Analytics & Reporting

- Active plans count
- Total revenue projection based on pricing and subscriber volume
- Plan popularity and usage stats

Data Model

SubscriptionPlan Entity

Table Name: `subscription_plans`

Field	Type	Required	Constraints	Description
<code>id</code>	Long	Auto	PK	Unique plan identifier
<code>title</code>	String(150)	✓	Unique	Plan display name
<code>description</code>	String(500)	✗	Max 500 chars	Short plan overview
<code>price</code>	BigDecimal(10, 2)	✓	>= 0	Plan subscription price
<code>durationInDays</code>	Integer	✓	>= 1	Validity period of plan in days
<code>billingCycle</code>	ENUM(MONTHLY/YEARLY/ONCE_TIME)	✗	Default = MONTHLY	Defines recurrence interval
<code>trialDays</code>	Integer	✗	>=0	Free access days before billing
<code>maxUsersAllowed</code>	Integer	✗	>=1	User limit allowed per subscription
<code>features</code>	JSON / TEXT	✗	List of feature tags	Benefits included in the plan
<code>autoRenew</code>	Boolean	✗	Default: true	Recurring subscription toggle
<code>active</code>	Boolean	! Required	Default: true	Soft delete / status flag
<code>createdAt</code>	LocalDateTime	Auto	Immutable	Creation timestamp
<code>updatedAt</code>	LocalDateTime	Auto	Updated on change	Last update timestamp

Relationships

Relationship	Description
Subscription Plan → UserSubscription	One plan can have multiple subscribed users

API Endpoints

Base URL

`/api/subscription-plans`

1. Create a Subscription Plan

Endpoint: `POST /api/subscription-plans`

Description: Adds a new subscription plan to system.

Request Body

```
{ "title": "Premium Membership", "description": "Full access to features with priority support and unlimited usage.", "price": 999.00, "durationInDays": 30, "billingCycle": "MONTHLY", "trialDays": 7, "maxUsersAllowed": 1, "features": ["Unlimited Access", "Priority Support", "Advanced Analytics"], "autoRenew": true, "active": true}
```

2. Bulk Create Plans

Endpoint: `POST /api/subscription-plans/bulk`

Description: Create multiple subscription packages at once.

```
[ { "title": "Basic Plan", "price": 199.00, "durationInDays": 30, "billingCycle": "MONTHLY", "features": ["Limited Access"] }, { "title": "Gold Annual Plan", "price": 4999.00, "durationInDays": 365, "billingCycle": "YEARLY", "trialDays": 14, "features": ["Unlimited Access", "Advanced Support", "Free Updates"] } ]
```

3. Get Plan by ID

Endpoint: `GET /api/subscription-plans/{id}`

Example: `GET /api/subscription-plans/1`

4. Update Subscription Plan

Endpoint: `PUT /api/subscription-plans/{id}`

Example Body

```
{ "title": "Premium Plus Membership", "price": 1299.00, "durationInDays": 30, "billingCycle": "MONTHLY", "features": ["Unlimited Access", "VIP Support", "Custom Reports"], "trialDays": 10 }
```

5. Soft Delete Plan

Endpoint: `DELETE /api/subscription-plans/{id}`

Response

```
{ "message": "Subscription plan deactivated successfully", "success": true }
```

Note: The plan will not be shown in new listings.

6. Permanent Delete Plan

Endpoint: `DELETE /api/subscription-plans/{id}/permanent`

```
{ "message": "Plan permanently deleted", "success": true }
```

⚠ Irreversible action.

7. List & Filter Plans

Endpoint: `GET /api/subscription-plans`

Filters include:

Query Param	Example	Description
<code>active</code>	true/false	Filter active/inactive
<code>billingCycle</code>	MONTHLY	Filtering by billing type
<code>minPrice</code> & <code>maxPrice</code>	99 → 999	Price range filter

Validation Rules

Required Fields

- Title (unique, max length 150)
- Price (numeric, ≥ 0)
- Duration in days (min 1)
- Active status

Optional Fields

- Trial days (≥ 0)
- Features list
- Billing type (default = MONTHLY)
- Auto renew flag

Business Constraints

- ▼ **Watchlist**
- 1. Duplicate plan titles are not allowed.

2. Trial period must not exceed plan validity.

3. Soft-deleted plans can be restored by setting `active=true`

Wishlist Module Documentation

Error Handling Overview

Status	Cause
400 Bad Request	Invalid input or missing fields
404 Not Found	Subscription plan not available
409 Conflict	Duplicate plan title

The **Wishlist Module** enables users to save books they are interested in for future reference. A wishlist serves as a personalized bookmark system where users can store books, add notes, revisit later, and track books they intend to read or purchase.

This module supports CRUD operations including adding/removing wishlist items, updating notes, pagination listing, and wishlist analytics (count, item presence check).

Example

Features

```
{ "message": "Plan with title 'Premium Membership' already exists", "success": false }
```

1. Wishlist Management

- Add books to wishlist
- Remove items individually
- Update personal notes for saved books
- Prevent duplicate wishlist entries via unique constraint

2. User Based Storage & Access Control

- Each wishlist entry is associated with a specific user
- Supports role-based access: USER and ADMIN
- Admins can fetch wishlist details of any user

3. Advanced Fetching & Pagination

- Fetch logged-in user's wishlist with pagination
- Fetch wishlist of any user (Admin/Public applicable)
- Efficient database-level indexing for fast queries

4. Wishlist Insights & Analytics

- Count wishlist items of current user
- Count how many users wishlisted a specific book
- Check if a book exists in user's wishlist

Data Model

Wishlist Entity

Table Name: `wishlists`

Field	Type	Required	Constraints	Description
<code>id</code>	Long	Auto	PK	Unique wishlist entry ID
<code>user</code>	User	✓	FK, Not Null	Owner of the wishlist item
<code>book</code>	Book	✓	FK, Not Null	Book added to wishlist
<code>addedAt</code>	LocalDateTime	Auto	Immutable	Timestamp when saved
<code>notes</code>	String(500)	✗	Max 500 chars	Optional user notes

Indexes & Constraints

Type	Columns	Purpose
Unique Constraint	<code>user_id</code> , <code>book_id</code>	Prevents duplicate wishlist entries
Index	<code>user_id</code>	Optimized retrieval for user-wise wishlist
Index	<code>book_id</code>	Faster book wishlist analytics

Relationships

Relationship	Type	Description
<code>Wishlist → User</code>	Many-to-One	A user can have multiple wishlist items
<code>Wishlist → Book</code>	Many-to-One	A book may be wishlisted by many users

API Endpoints

Base URL

`/api/wishlist`

1. Add Book to Wishlist

Endpoint: `POST /api/wishlist/add/{bookId}`

Authorization: USER / ADMIN

Query Params:

`notes` → Optional notes describing reason for saving

Example Request

```
POST /api/wishlist/add/10?notes=Read after finals
```

Success Response: `201 Created`

Returns `WishlistDTO`

2. Remove Book from Wishlist

Endpoint: `DELETE /api/wishlist/remove/{bookId}`

Authorization: USER / ADMIN

Example Request


```
DELETE /api/wishlist/remove/10
```

Success Response

```
{ "message": "Book removed from wishlist successfully", "success": true }
```

3. Update Wishlist Notes

Endpoint: `PUT /api/wishlist/update-notes/{bookId}`

Authorization: USER / ADMIN

Query Param:

`notes` → New updated notes

Example Request

```
PUT /api/wishlist/update-notes/10?notes=High priority read
```

Success Response: `200 OK`

Returns updated `WishlistDTO`

4. Get Logged-In User Wishlist (Paginated)

Endpoint: `GET /api/wishlist/my-wishlist`

Authorization: USER / ADMIN

Query Params:

Param	Default	Description
page	0	Page number
size	10	Items per page

Example

```
GET /api/wishlist/my-wishlist?page=0&size=5
```

Returns `PageResponse<WishlistDTO>`

5. Get Wishlist of Any User

Endpoint: `GET /api/wishlist/user/{userId}`

Authorization: Public/Admin (Based on use case)

```
GET /api/wishlist/user/5?page=0&size=10
```

Returns paginated `PageResponse<WishlistDTO>`

6. Check if Book is Wishlisted

Endpoint: `GET /api/wishlist/check/{bookId}`

Authorization: USER / ADMIN

Response Example

```
{ "isInWishlist": true }
```

7. Get Logged-In User Wishlist Count

Endpoint: `GET /api/wishlist/my-count`

Authorization: USER / ADMIN

Response

```
{ "count": 18 }
```

8. Get Count of Users Who Wishlisted a Book

Endpoint: `GET /api/wishlist/book/{bookId}/count`

Example Response

```
{ "count": 42 }
```

Validation Rules

Mandatory Fields

- `user` must exist and be authenticated
- `book` must be valid and present in catalog
- Duplicate wishlist entries not allowed (unique constraint)

Optional Fields

- notes (max length 500)

Business Rules

Rule	Explanation
No Duplicate Wishlisting	A user cannot add the same book twice
Auto Timestamp	<code>addedAt</code> generated automatically

Error Handling

Status	Reason
400 Bad Request	Invalid data, duplicate book in wishlist
404 Not Found	User/Book not found or wishlist empty
401 Unauthorized	User not authenticated

Example Bad Request

```
{ "message": "Book already exists in wishlist", "success": false }
```

▼ User



User Module Documentation



Overview

The **User Module** manages the system's registered users, storing authentication details, roles, login provider, timestamps, and profile information.

This module provides endpoints to fetch user details, list all users, and retrieve authenticated user profiles via JWT.

It also includes an **Admin-only analytics feature** to fetch total registered users.



Features

1. User Authentication Data Storage

- Stores OAuth (Google) and manual registered users
- Supports password-based & third-party login (`AuthProvider`)
- Maintains verification status for access privileges

2. Profile & Identity Details

- Stores personal info → Full name, email, phone, profile image
- Tracks user login history using `lastLogin`
- Email is unique across the platform

3. Role-Based Authorization

- Supports multiple user roles via `UserRole`
- Used for access control across protected resources

4. Account Lifecycle Tracking

- Auto timestamps for account creation & updates
- Tracks last login event for analytics



Data Model

User Entity

Table Name: `users`

Field	Type	Required	Constraints	Description
<code>id</code>	Long	Auto	PK	Unique identifier
<code>fullName</code>	String	✓	Not Blank	User's full display name
<code>password</code>	String	🔒 Internal	Hidden via <code>@JsonIgnore</code>	Used only for auth (local accounts)
<code>email</code>	String	✓	Unique, Not Blank, Valid Email	Primary login identifier
<code>phone</code>	String	✗	None	Optional contact number
<code>authProvider</code>	Enum	Auto	Default: LOCAL	Indicates signup source (LOCAL/GOOGLE)
<code>googleId</code>	String	✗	Null for local users	OAuth reference ID
<code>profileImage</code>	String	✗	None	Profile picture URL
<code>role</code>	Enum(UserRole)	🔥 Required	NotNull	Defines user permissions (ADMIN/USER)
<code>verified</code>	Boolean	Auto	Default: false	Email/Account verification status
<code>createdAt</code>	LocalDateTime	Auto	Immutable	Account creation timestamp
<code>updatedAt</code>	LocalDateTime	Auto	Auto-Update	Last profile update time
<code>lastLogin</code>	LocalDateTime	✗	None	Updated on login event

Relationships

This entity stands independently.

Other modules refer to `User` for ownership (orders, wishlist, reviews, etc.)

API Endpoints

Base URL

`/api/users`

1. Get Profile From JWT Token

 *Authenticated Users Only*

Endpoint: `GET /api/users/profile`

Headers

Key	Value
Authorization	Bearer Token

Response (200 OK)

```
{ "id": 14, "fullName": "John Carter", "email": "john@gmail.com", "role": "USER", "profileImage": null, "verified": true, "lastLogin": "2025-02-01T12:40:19" }
```

2. Get List of All Users

 *Public or Admin Based on Security Layer*

Endpoint: `GET /users/list`

Returns raw list of `User` entities.

3. Get User By ID

Endpoint: `GET /users/{userId}`

Fetches profile of any specific user.

Example

```
GET /users/8
```

Returns a single `UserDTO`

4. Get Total User Statistics (Admin Only)

Endpoint: `GET /api/users/statistics`

Authorization: `ADMIN`

Sample Response

```
{ "totalUsers": 245 }
```

Validation Rules

Field	Rule
fullName	Required, cannot be empty
email	Must be valid email format & unique
role	Must be assigned manually or by signup
password	Stored hidden, only for internal auth

Error Handling

▼ Payment

Status	Cause
400 Bad Request	Invalid email/role provided
401 Unauthorized	API key missing or invalid
403 Forbidden	Non-admin trying to access /statistics
404 Not Found	User not found by ID

Payment Module Documentation

Overview

The **Payment Module** handles all financial transactions within the system including fines, membership subscriptions, penalties, overdue charges, and

Example Error Response

It manages the complete lifecycle of a payment — from **initiation** → **verification** → **retry** → **completion** and provides admin-level financial statistics & payment history.

The module supports **multiple payment gateways (Razorpay, Stripe)** and ensures secure verification through transaction signatures and gateway identifiers.

🎯 Key Features

1. Multi-Gateway Payment Support

- Supported gateways: **RAZORPAY**, **STRIPE**
- Stores gateway IDs, signatures, order IDs, metadata

2. Complete Payment Lifecycle

- Status progression: **PENDING** → **PROCESSING** → **SUCCESS/FAILED/CANCELLED**
- Verification with signature validation
- Retry system for failed/cancelled transactions

3. Fine + Subscription Linked Payments

- Payments can be associated with either:
 - Book Loan (Fines)
 - Subscription (Membership/Plans)
 - Generic Fine entity

4. Tracking & Analytics

- Revenue reports for current month (Admin)
 - Sorted and paginated retrieval of user payments
 - Financial logging for audit trail
-

Data Model

Payment Entity

Table Name: `payments`

Field	Type	Required	Description
<code>id</code>	Long	Auto	Unique payment record ID